

# Informatika 3

Prednáška 7: Kódovanie reťazcov

Michal Mrena



ŽILINSKÁ UNIVERZITA V ŽILINE  
Fakulta riadenia  
a informatiky

# Kódovanie reťazcov



# Kódovanie reťazcov

- Za reťazec považujeme ľubovoľnú postupnosť znakov
- Každému znaku priradujeme číselnú hodnotu (angl. Code Point)
- Hodnoty sú určené štandardom
- V prednáške predstavíme štandardy:
  - ASCII (z angl. American Standard Code for Information Interchange)
  - Rozšírenia ASCII
  - Unicode

```
double (*(arr[5])(int))(int);
```

```
[](){}();
```

Cormar nelde Eldatárin nu Tarmenel.

# Štandard ASCII

- Obsahuje 128 znakov
- Na zakódovanie jedného znaku je potrebných 7 bitov
- Jej obsah ovplyvnil syntax programovacích jazykov
- **Neobsahuje** znaky národných abecied

| Dec | Hx | Oct | Char                        | Dec | Hx | Oct | Html  | Chr   | Dec | Hx | Oct | Html  | Chr | Dec | Hx | Oct | Html   | Chr |
|-----|----|-----|-----------------------------|-----|----|-----|-------|-------|-----|----|-----|-------|-----|-----|----|-----|--------|-----|
| 0   | 0  | 000 | NUL (null)                  | 32  | 20 | 040 | &#32; | Space | 64  | 40 | 100 | &#64; | @   | 96  | 60 | 140 | &#96;  | `   |
| 1   | 1  | 001 | SOH (start of heading)      | 33  | 21 | 041 | &#33; | !     | 65  | 41 | 101 | &#65; | A   | 97  | 61 | 141 | &#97;  | a   |
| 2   | 2  | 002 | STX (start of text)         | 34  | 22 | 042 | &#34; | "     | 66  | 42 | 102 | &#66; | B   | 98  | 62 | 142 | &#98;  | b   |
| 3   | 3  | 003 | ETX (end of text)           | 35  | 23 | 043 | &#35; | #     | 67  | 43 | 103 | &#67; | C   | 99  | 63 | 143 | &#99;  | c   |
| 4   | 4  | 004 | EOT (end of transmission)   | 36  | 24 | 044 | &#36; | \$    | 68  | 44 | 104 | &#68; | D   | 100 | 64 | 144 | &#100; | d   |
| 5   | 5  | 005 | ENQ (enquiry)               | 37  | 25 | 045 | &#37; | %     | 69  | 45 | 105 | &#69; | E   | 101 | 65 | 145 | &#101; | e   |
| 6   | 6  | 006 | ACK (acknowledge)           | 38  | 26 | 046 | &#38; | &     | 70  | 46 | 106 | &#70; | F   | 102 | 66 | 146 | &#102; | f   |
| 7   | 7  | 007 | BEL (bell)                  | 39  | 27 | 047 | &#39; | '     | 71  | 47 | 107 | &#71; | G   | 103 | 67 | 147 | &#103; | g   |
| 8   | 8  | 010 | BS (backspace)              | 40  | 28 | 050 | &#40; | (     | 72  | 48 | 110 | &#72; | H   | 104 | 68 | 150 | &#104; | h   |
| 9   | 9  | 011 | TAB (horizontal tab)        | 41  | 29 | 051 | &#41; | )     | 73  | 49 | 111 | &#73; | I   | 105 | 69 | 151 | &#105; | i   |
| 10  | A  | 012 | LF (NL line feed, new line) | 42  | 2A | 052 | &#42; | *     | 74  | 4A | 112 | &#74; | J   | 106 | 6A | 152 | &#106; | j   |
| 11  | B  | 013 | VT (vertical tab)           | 43  | 2B | 053 | &#43; | +     | 75  | 4B | 113 | &#75; | K   | 107 | 6B | 153 | &#107; | k   |
| 12  | C  | 014 | FF (NP form feed, new page) | 44  | 2C | 054 | &#44; | ,     | 76  | 4C | 114 | &#76; | L   | 108 | 6C | 154 | &#108; | l   |
| 13  | D  | 015 | CR (carriage return)        | 45  | 2D | 055 | &#45; | -     | 77  | 4D | 115 | &#77; | M   | 109 | 6D | 155 | &#109; | m   |
| 14  | E  | 016 | SO (shift out)              | 46  | 2E | 056 | &#46; | .     | 78  | 4E | 116 | &#78; | N   | 110 | 6E | 156 | &#110; | n   |
| 15  | F  | 017 | SI (shift in)               | 47  | 2F | 057 | &#47; | /     | 79  | 4F | 117 | &#79; | O   | 111 | 6F | 157 | &#111; | o   |
| 16  | 10 | 020 | DLE (data link escape)      | 48  | 30 | 060 | &#48; | 0     | 80  | 50 | 120 | &#80; | P   | 112 | 70 | 160 | &#112; | p   |
| 17  | 11 | 021 | DC1 (device control 1)      | 49  | 31 | 061 | &#49; | 1     | 81  | 51 | 121 | &#81; | Q   | 113 | 71 | 161 | &#113; | q   |
| 18  | 12 | 022 | DC2 (device control 2)      | 50  | 32 | 062 | &#50; | 2     | 82  | 52 | 122 | &#82; | R   | 114 | 72 | 162 | &#114; | r   |
| 19  | 13 | 023 | DC3 (device control 3)      | 51  | 33 | 063 | &#51; | 3     | 83  | 53 | 123 | &#83; | S   | 115 | 73 | 163 | &#115; | s   |
| 20  | 14 | 024 | DC4 (device control 4)      | 52  | 34 | 064 | &#52; | 4     | 84  | 54 | 124 | &#84; | T   | 116 | 74 | 164 | &#116; | t   |
| 21  | 15 | 025 | NAK (negative acknowledge)  | 53  | 35 | 065 | &#53; | 5     | 85  | 55 | 125 | &#85; | U   | 117 | 75 | 165 | &#117; | u   |
| 22  | 16 | 026 | SYN (synchronous idle)      | 54  | 36 | 066 | &#54; | 6     | 86  | 56 | 126 | &#86; | V   | 118 | 76 | 166 | &#118; | v   |
| 23  | 17 | 027 | ETB (end of trans. block)   | 55  | 37 | 067 | &#55; | 7     | 87  | 57 | 127 | &#87; | W   | 119 | 77 | 167 | &#119; | w   |
| 24  | 18 | 030 | CAN (cancel)                | 56  | 38 | 070 | &#56; | 8     | 88  | 58 | 130 | &#88; | X   | 120 | 78 | 170 | &#120; | x   |
| 25  | 19 | 031 | EM (end of medium)          | 57  | 39 | 071 | &#57; | 9     | 89  | 59 | 131 | &#89; | Y   | 121 | 79 | 171 | &#121; | y   |
| 26  | 1A | 032 | SUB (substitute)            | 58  | 3A | 072 | &#58; | :     | 90  | 5A | 132 | &#90; | Z   | 122 | 7A | 172 | &#122; | z   |
| 27  | 1B | 033 | ESC (escape)                | 59  | 3B | 073 | &#59; | ;     | 91  | 5B | 133 | &#91; | [   | 123 | 7B | 173 | &#123; | {   |
| 28  | 1C | 034 | FS (file separator)         | 60  | 3C | 074 | &#60; | <     | 92  | 5C | 134 | &#92; | \   | 124 | 7C | 174 | &#124; |     |
| 29  | 1D | 035 | GS (group separator)        | 61  | 3D | 075 | &#61; | =     | 93  | 5D | 135 | &#93; | ]   | 125 | 7D | 175 | &#125; | }   |
| 30  | 1E | 036 | RS (record separator)       | 62  | 3E | 076 | &#62; | >     | 94  | 5E | 136 | &#94; | ^   | 126 | 7E | 176 | &#126; | ~   |
| 31  | 1F | 037 | US (unit separator)         | 63  | 3F | 077 | &#63; | ?     | 95  | 5F | 137 | &#95; | _   | 127 | 7F | 177 | &#127; | DEL |

Source: [www.LookupTables.com](http://www.LookupTables.com)

# Rozšírenia ASCII

- Problém kódovania národných abecied (diakritiky) je riešený pridaním ďalšie bitu, čím sa získa ďalších 128 hodnôt
- Tieto sú použité na zakódovanie znakov špecifických pre rôzne jazyky
- Jedno rozšírenie však nedokáže pokryť všetky jazyky, preto ich existuje niekoľko

|     |   |     |   |     |   |     |   |     |   |     |   |     |   |     |   |
|-----|---|-----|---|-----|---|-----|---|-----|---|-----|---|-----|---|-----|---|
| 128 | Ç | 144 | É | 160 | á | 176 | ☐ | 192 | Ł | 208 | ⌚ | 224 | α | 240 | ≡ |
| 129 | ü | 145 | æ | 161 | í | 177 | ☐ | 193 | ± | 209 | ⌚ | 225 | β | 241 | ± |
| 130 | é | 146 | Æ | 162 | ó | 178 | ☐ | 194 | ⌚ | 210 | ⌚ | 226 | Γ | 242 | ≥ |
| 131 | â | 147 | ô | 163 | ú | 179 |   | 195 | ⌚ | 211 | ⌚ | 227 | π | 243 | ≤ |
| 132 | ä | 148 | ö | 164 | ñ | 180 | ⌚ | 196 | — | 212 | ⌚ | 228 | Σ | 244 | ∫ |
| 133 | à | 149 | ò | 165 | Ñ | 181 | ⌚ | 197 | + | 213 | ⌚ | 229 | σ | 245 | ∫ |
| 134 | â | 150 | û | 166 | ² | 182 | ⌚ | 198 | ⌚ | 214 | ⌚ | 230 | μ | 246 | + |
| 135 | ç | 151 | ù | 167 | ° | 183 | ⌚ | 199 | ⌚ | 215 | ⌚ | 231 | τ | 247 | ≈ |
| 136 | ê | 152 | ÿ | 168 | ¿ | 184 | ⌚ | 200 | ⌚ | 216 | ⌚ | 232 | Φ | 248 | ° |
| 137 | ë | 153 | Ö | 169 | ⌚ | 185 | ⌚ | 201 | ⌚ | 217 | ⌚ | 233 | Θ | 249 | · |
| 138 | è | 154 | Û | 170 | ⌚ | 186 | ⌚ | 202 | ⌚ | 218 | ⌚ | 234 | Ω | 250 | · |
| 139 | ï | 155 | ° | 171 | ½ | 187 | ⌚ | 203 | ⌚ | 219 | ■ | 235 | δ | 251 | √ |
| 140 | î | 156 | £ | 172 | ¾ | 188 | ⌚ | 204 | ⌚ | 220 | ■ | 236 | ∞ | 252 | Δ |
| 141 | ì | 157 | ¥ | 173 | ¡ | 189 | ⌚ | 205 | = | 221 | ■ | 237 | φ | 253 | ² |
| 142 | Ä | 158 | £ | 174 | « | 190 | ⌚ | 206 | ⌚ | 222 | ■ | 238 | ε | 254 | ■ |
| 143 | Å | 159 | ƒ | 175 | » | 191 | ⌚ | 207 | ± | 223 | ■ | 239 | ∩ | 255 |   |

Source: [www.LookupTables.com](http://www.LookupTables.com)

# Rozšírenia ASCII

- Najznámejším rozšírením je [Windows-1252](#) používaným v OS Windows
  - Veľmi podobný štandardu [ISO-8859-1](#) (Latin 1)
- Ďalšie, podobné, rozšírenia používané v našom regióne sú:
  - [Windows-1250](#)
  - [Windows-1257](#)
- Výhodou týchto rozšírení je, že jeden znak (**code point**) dokážeme reprezentovať jedným bajtom (**code unit**) – angl. **single byte encoding**

Windows-1252 (CP1252)<sup>[22][23][24][25][26]</sup>

|    | 0         | 1         | 2         | 3         | 4         | 5         | 6         | 7         | 8         | 9         | A         | B         | C         | D   | E         | F         |
|----|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----|-----------|-----------|
| 0_ | NUL       | SOH       | STX       | ETX       | EOT       | ENQ       | ACK       | BEL       | BS        | HT        | LF        | VT        | FF        | CR  | SO        | SI        |
| 1_ | DLE       | DC1       | DC2       | DC3       | DC4       | NAK       | SYN       | ETB       | CAN       | EM        | SUB       | ESC       | FS        | GS  | RS        | US        |
| 2_ | SP        | !         | "         | #         | \$        | %         | &         | '         | (         | )         | *         | +         | ,         | -   | .         | /         |
| 3_ | 0         | 1         | 2         | 3         | 4         | 5         | 6         | 7         | 8         | 9         | :         | ;         | <         | =   | >         | ?         |
| 4_ | @         | A         | B         | C         | D         | E         | F         | G         | H         | I         | J         | K         | L         | M   | N         | O         |
| 5_ | P         | Q         | R         | S         | T         | U         | V         | W         | X         | Y         | Z         | [         | \         | ]   | ^         | _         |
| 6_ | `         | a         | b         | c         | d         | e         | f         | g         | h         | i         | j         | k         | l         | m   | n         | o         |
| 7_ | p         | q         | r         | s         | t         | u         | v         | w         | x         | y         | z         | {         |           | }   | ~         | DEL       |
| 8_ | €<br>20AC |           | ‚<br>201A | ƒ<br>0192 | „<br>201E | …<br>2026 | †<br>2020 | ‡<br>2021 | ˆ<br>02C6 | %<br>2030 | Š<br>0160 | ‹<br>2039 | Œ<br>0152 |     | Ž<br>017D |           |
| 9_ |           | ‘<br>2018 | ’<br>2019 | “<br>201C | ”<br>201D | •<br>2022 | —<br>2013 | —<br>2014 | ~<br>02DC | ™<br>2122 | š<br>0161 | ›<br>203A | œ<br>0153 |     | ž<br>017E | ÿ<br>0178 |
| A_ | NBSP      | ı         | ¢         | £         | ¤         | ¥         | ¦         | §         | ¨         | ©         | ª         | «         | ¬         | SHY | ®         | ¯         |
| B_ | °         | ±         | ²         | ³         | ´         | µ         | ¶         | ·         | ¸         | ¹         | º         | »         | ¼         | ½   | ¾         | ¿         |
| C_ | À         | Á         | Â         | Ã         | Ä         | Å         | Æ         | Ç         | È         | É         | Ê         | Ë         | Ì         | Í   | Î         | Ï         |
| D_ | Ð         | Ñ         | Ò         | Ó         | Ô         | Õ         | Ö         | ×         | Ø         | Ù         | Ú         | Û         | Ü         | Ý   | Þ         | ß         |
| E_ | à         | á         | â         | ã         | ä         | å         | æ         | ç         | è         | é         | ê         | ë         | ì         | í   | î         | ï         |
| F_ | ð         | ñ         | ò         | ó         | ô         | õ         | ö         | ÷         | ø         | ù         | ú         | û         | ü         | ý   | þ         | ÿ         |

Zdroj: [Wikipedia](#)

# Štandard Unicode

- Na zakódovanie jedného znaku používa 21 bitov
- Obsahuje 1 112 064 znakov (code points), pričom prvých 128 znakov je rovnakých ako v štandarde ASCII
- Znaký typicky označujeme hexadecimálnymi číslami U+0000 až U+0010 FFFF
- Na kódovanie Unicode znakov používame jedno z kódovaní:
  - [UTF-8](#)
  - [UTF-16](#)
  - [UTF-32](#)



U+1F996



U+1F43C

# UTF-8

- Na zakódovanie jedného znaku (code point) používa 1 až 4 bajty (code unit) – kódovanie s variabilnou dĺžkou (angl. variable length encoding)
  - znaky anglickej abecedy – 1 bajt
  - ďalšie európske znaky – 2 bajty
  - ďalšie znaky a symboly – 4 bajty
- Prakticky štandard používaný na internete
- Používaný v OS Linux

```
<!DOCTYPE html>
<html xmlns="..." xml:lang="sk-sk" lang="sk-sk" dir="ltr">
<head>
  <meta charset="utf-8">
  <meta name="robots" content="index, follow">

  <title>Žilinská univerzita v Žiline</title>
</head>
```

```
mrena3@frios2:~$ echo $LANG
en_US.UTF-8
mrena3@frios2:~$ |
```



# UTF-8

- Na uloženie počtu bajtov pre jeden znak používa **kontrolné bity**
- Kód znaku získame spojením **ostatných bitov**
- Spätne kompatibilný s kódovaním ASCII

0xxxxxxx  
110xxxxx 10xxxxxx  
1110xxxx 10xxxxxx 10xxxxxx  
11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

11110000 10011111 10010000 10111100  
000 011111 010000 111100

128060<sub>10</sub>

1F43C<sub>16</sub>

U+1F43C



# UTF-16

- Na zakódovanie 1 znaku (code point) používa 1 až 2 dvojbyty (code unit) – kódovanie s variabilnou dĺžkou
- Používa ho rozhranie Windows API
- Interne ho používajú napríklad jazyky Java (typ `char` zaberá 2 bajty) a JavaScript\*, z časti aj Python
- Nie je spätne kompatibilný s kódovaním ASCII

# UTF-16

- Znak **U+0000** až **U+D7FF** a **U+E000** až **U+FFFF** kóduje jedným dvojбайтом zodpovedajúcim hodnote znaku
- Znak **U+010000** až **U+10FFFF** kóduje dvoma dvojбайtami
  - Od hodnoty znaku odpočítame číslo 0x10000
  - K horným 10 bitom pripočítame kontrolné bity **0xD800** – dostaneme prvý dvojбайt
  - K dolným 10 bitom pripočítame kontrolné bity **0xDC00** – dostaneme druhý dvojбайt



U+1F43C

$$1F43C_{16} - 10000_{16} = F43C_{16}$$

$$F43C_{16} = 00001111010000111100_2$$

$$D800_{16} + 3D_{16}$$

$$D83D_{16}$$

$$1101100000111101$$

$$DC00_{16} + 3C_{16}$$

$$DC3C_{16}$$

$$1101110000111100$$

# UTF-32

- Na zakódovanie jedného znaku (code point) používa 1 štvorbajt (code unit)
- Toto kódovanie je pamäťovo neefektívne
- Na druhej strane je práca s ním jednoduchá
- Zvyčajne sa používa na (dočasnú) reprezentáciu textu v pamäti

# Reťazce v jazyku C++



# Reprezentácia znakov

- Jazyk C++ neobsahuje vstavanú podporu konkrétneho kódovania
- Obsahuje však nástroje na prácu s rôznymi kódovaniami
- Na reprezentáciu (častí) znakov (code unit) primárne používame typy `char` a `wchar_t`
  - `char` zaberá 1 bajt\* – môže byť použitý na reprezentáciu reťazcov v kódovaní ASCII alebo UTF-8
  - `wchar_t` zaberá
    - 2 bajty v systéme *Windows* – môže byť použitý na reprezentáciu reťazcov v kódovaní UTF-16
    - 4 bajty v systéme *Linux* – môže byť použitý na reprezentáciu reťazcov v kódovaní UTF-32

Takmer na všetkých platformách zaberá 8 bitov. \*

# Reprezentácia znakov

- Ďalej obsahuje nasledovné typy:
  - (od C++20) `char8_t` – typ určený na reprezentáciu code units v kódovaní UTF-8
  - (od C++11) `char16_t` – typ určený na reprezentáciu code units v kódovaní UTF-16
  - (od C++11) `char32_t` – typ určený na reprezentáciu code units v kódovaní UTF-32
- Na rozdiel od typov `char` a `wchar_t` majú pevne danú veľkosť – ich použitie je platformovo nezávislé

Pre jednoduchosť budeme na nasledujúcich snímkach predpokladať prácu s textom obsahujúcim iba znaky (code point) reprezentovateľné jedným code unit typu `char`.

# Reťazce v jazyku C

- Pri práci s reťazcami je jazyk C++ ovplyvnený svojimi koreňmi v jazyku C
- V jazyku C za reťazec považujeme pole znakov ukončené znakom s hodnotou 0 (literál typu char reprezentujúci túto hodnotu zapisujeme syntaxou `'\0'`)
- Reťazec `"Hello World!"` by tak bol reprezentovaný poľom o veľkosti 13
- Pole znakov je zároveň typom literálu (posledný znak `'\0'` je pridaný automaticky)

```
main.cpp x
main.cpp > ...
4
3 int main() {
2     const char hello[] = "Hello World!";
1 }
5
```

**expression**  
Type: `const char[13]`  
Value = `{72, 101, 108, 108, 111, 32, 87, 111, 114, 108, ...}`



# Reťazce v jazyku C

- Typ pole je implicitne konvertovateľný na ukazovateľ na začiatok poľa (na prvý prvok poľa)
- Funkcie pracujúce s reťazcami zvyčajne preberajú ukazovateľ na typ (`const`) `char` – ukazovateľ na začiatok poľa
- Tieto funkcie sa spoliehajú na to, že je pole ukončené znakom `'\0'`
- Tento prístup je pomerne náchylný na chyby

```
#include <iostream>

int strlen(const char *s) {
    int len = 0;
    while (*s != '\0') {
        ++len;
        ++s;
    }
    return len;
}

int main() {
    const char hello[] = "Hello World!";
    std::cout << strlen(hello) << "\n";
}
```

# Reťazce v jazyku C++

- Základným typom na prácu s reťazcami v jazyku C++ je trieda `std::string`
- Na pozadí trieda spravuje pole znakov – reťazec v štýle jazyka C
- Jej rozhranie, konštruktory a deštruktor však umožňujú oveľa bezpečnejšiu prácu s reťazcom
- *Konverzný* konštruktor zabezpečuje inicializáciu z reťazca jazyka C (vrátane literálu)

```
int main() {  
    const char hello[] = "Hello World!";  
    std::string s(hello);  
    std::cout << s.size() << "\n";  
}
```

```
int main() {  
    const char hello[] = "Hello World!";  
    std::string s(hello);  
    s += " And Hello Again!";  
    std::cout << s << "\n";  
    // Hello World! And Hello Again!  
}
```

Trieda `std::string` je definovaná v hlavičke `<string>`.

Na rozdiel od jazyka Java je možné inštancie triedy `string` modifikovať.

# Trieda `std::string`

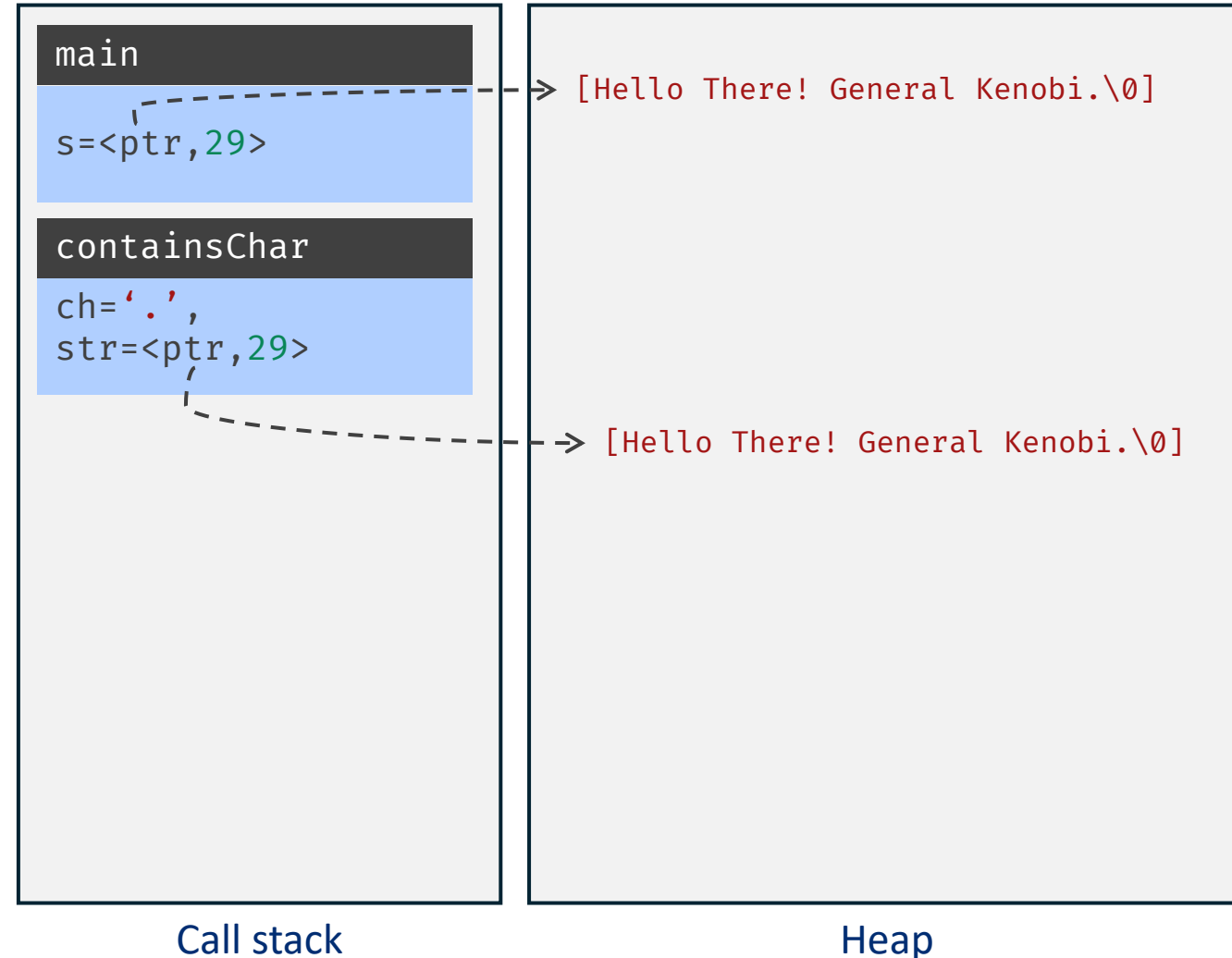
- Trieda `string` uchováva ukazovateľ na začiatok poľa znakov a jeho veľkosť
- V konštruktore (11) pole alokuje a inicializuje kópiou parametra
- Kopírovací konštruktor (13) zabezpečí vytvorenie kópie poľa
- Pri modifikácii (12) zabezpečí alokovanie väčšie poľa
- V deštruktore (16) pole dealokuje

```
1 bool containsChar(std::string str, char ch) {
2     for (size_t i = 0; i < str.size(); ++i) {
3         if (str[i] == ch) {
4             return true;
5         }
6     }
7     return false;
8 }
9
10 int main() {
11     std::string s("Hello There!");
12     s += " General Kenobi.";
13     if (containsChar(s, '.')) {
14         std::cout << "It has .\n";
15     }
16 }
```

# Trieda `std::string`

```
1 bool containsChar(std::string str, char ch) {  
2     for (size_t i = 0; i < str.size(); ++i) {  
3         if (str[i] == ch) {  
4             return true;  
5         }  
6     }  
7     return false;  
8 }  
9  
10 int main() {  
11     std::string s("Hello There!");  
12     s += " General Kenobi.";  
13     if (containsChar(s, '.')) {  
14         std::cout << "It has .\n";  
15     }  
16 }
```

Pri práci s reťazcami sa chceme  
vyhnúť **zbytočným kópiám!**



# Trieda `std::string`

```
1 bool containsChar(const std::string &str, char ch) {
2     for (size_t i = 0; i < str.size(); ++i) {
3         if (str[i] == ch) {
4             return true;
5         }
6     }
7     return false;
8 }
9
10 int main() {
11     std::string s("Hello There!");
12     s += " General Kenobi.";
13     if (containsChar(s, '.')) {
14         std::cout << "It has .\n";
15     }
16 }
```

# Kopírovanie reťazcov

Častou chybou je vytváranie  
zbytočných kópií v get metódach  
a pri inicializácii

```
class Person {  
public:  
    Person(std::string name);  
    std::string getName() const;  
  
private:  
    std::string m_name;  
};  
  
Person::Person(std::string name) :  
    m_name(name) {  
}  
  
std::string Person::getName() const {  
    return m_name;  
}
```

# Kopírovanie reťazcov

- Základné riešenie je použitie odkazov
- Výhodné je obzvlášť pri get metódach
- Pri inicializácii však v niektorých situáciách vznikne viac kópií, ako je nutné

```
int main() {  
    Person p("Larry");  
    std::cout << p.getName() << "\n";  
}
```

```
class Person {  
public:  
    Person(const std::string &name);  
    const std::string &getName() const;  
  
private:  
    std::string m_name;  
};  
  
Person::Person(const std::string &name) :  
    m_name(name) {  
}  
  
const std::string &Person::getName() const {  
    return m_name;  
}
```

# Kopírovanie reťazcov

Jednej kópii sa môžeme vyhnúť použitím **move konštruktora**, v ktorom si (vznikajúca) členská premenná `m_name` preberie pole zo (čoskoro zanikajúceho parametra) `name`.

```
int main() {  
    Person p("Larry");  
    std::cout << p.getName() << "\n";  
}
```

```
class Person {  
public:  
    Person(std::string name);  
    const std::string &getName() const;  
  
private:  
    std::string m_name;  
};  
  
Person::Person(std::string name) :  
    m_name(std::move(name)) {}  
  
const std::string &Person::getName() const {  
    return m_name;  
}
```



# Kopírovanie reťazcov

Ak (členská) funkcia nadobúda vlastníctvo je lepšie prebrať parameter ako hodnotu a potom inicializovať vlastníka move konštruktorom.

O tom, či sa vytvorí najviac jedna kópia rozhoduje ten, kto funkciu volá.

```
class Person {
public:
    Person(std::string name);
    const std::string &getName() const;

private:
    std::string m_name;
};

Person::Person(std::string name) :
    m_name(std::move(name)) {

const std::string &Person::getName() const {
    return m_name;
}
```

# Kopírovanie reťazcov

```
int main() {  
    // 1x konverzia 1x move  
    Person p1("Larry");  
  
    // 1x kópia a 1x move  
    std::string s1("Gibbs");  
    Person p2(s1);  
  
    // 2x move  
    std::string s2("Hector");  
    Person p3(std::move(s2));  
}
```

```
class Person {  
public:  
    Person(std::string name);  
    const std::string &getName() const;  
  
private:  
    std::string m_name;  
};  
  
Person::Person(std::string name) :  
    m_name(std::move(name)) {  
}  
  
const std::string &Person::getName() const {  
    return m_name;  
}
```

# Kopírovanie reťazcov

Ani použitie odkazov negarantuje vyhnutie sa kópiám alebo nepotrebným alokáciám pamäte.

V príklade vznikne dočasná inštancia triedy `string` (inicializovaná podľa literálu), na ktorú sa viaže odkaz `str`.

Lietrál je však dostupný počas celého behu programu.

```
bool containsChar(const std::string &str, char ch) {  
    for (const char c : str) {  
        if (c == ch) {  
            return true;  
        }  
    }  
    return false;  
}  
  
int main() {  
    if (containsChar("I'll be back!", '!')) {  
        std::cout << "He is back!\n";  
    }  
}
```

Cez reťazce môžeme iterovať cyklom `for-each`.

# Pohľad na reťazec

- (od C++17) Trieda `std::string_view` má rovnaké rozhranie ako trieda `std::string` avšak **nie je vlastníkom** poľa znakov
- Poskytuje iba **nemenný** pohľad na reťazec, ktorý vlastní niekto iný
- Inštancie pohľadu uchovávajú iba dva ukazovatele – ich kópia je lacná
- Preto ich posielame hodnotou
- `string_view` nie je implicitne konvertovateľný na `string`

```
bool containsChar(std::string_view str, char ch) {  
    for (const char c : str) {  
        if (c == ch) {  
            return true;  
        }  
    }  
    return false;  
}  
  
int main() {  
    if (containsChar("I'll be back!", '!')) {  
        std::cout << "He is back!\n";  
    }  
}
```

Trieda `std::string_view` je definovaná v hlavičke `<string_view>`.

# Pohľad na reťazec

V príklade vznikne iba inštancia triedy `string_view` – nie je potrebné žiadne alokovanie pamäte na halde.

Ak (členská) funkcia nepotrebuje nadobudnúť vlastníctvo parametra, je lepšie použiť typ `string_view`.

```
bool containsChar(std::string_view str, char ch) {  
    for (const char c : str) {  
        if (c == ch) {  
            return true;  
        }  
    }  
    return false;  
}  
  
int main() {  
    if (containsChar("I'll be back!", '!')) {  
        std::cout << "He is back!\n";  
    }  
}
```

Trieda `std::string_view` je definovaná v hlavičke `<string_view>`.

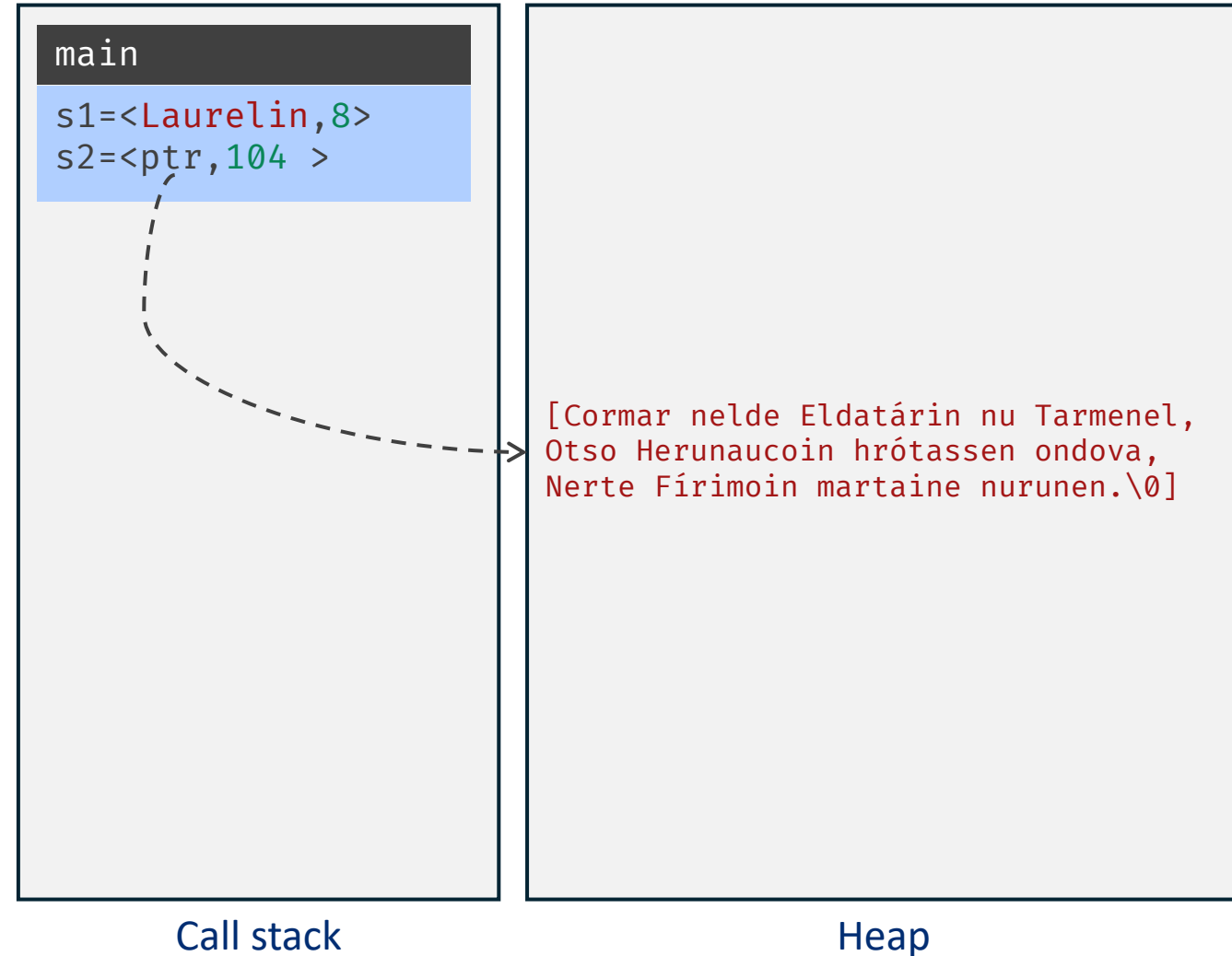
# Krátke reťazce

- Inštancie triedy `string` neobsahujú samotný reťazec
- Reťazec je uložený v dynamicky alokovanom poli na halde
- Väčšina implementácií štandardnej knižnice však v triede `string` implementuje optimalizáciu krátkych reťazcov (angl. short string optimization)
- Ak je reťazec krátky\* nedochádza k dynamickému alokovaniu pamäte, ale reťazec je súčasťou pamäťovej reprezentácie inštancie triedy `string`
- Práca s krátkymi reťazcami je tak podstatne efektívnejšia

Hranica krátkeho reťazca sa pre rôzne implementácie líši. Pre `libc++` je to 22 a `libstdc++` a `MSVC STL` je to 15. \*

# Krátke reťazce

```
int main() {  
    std::string s1("Laurelin");  
    std::string s2(  
        "Cormar nelde Eldatárin nu Tarmenel,"  
        "Otso Herunaucoin hrótassen ondova,"  
        "Nerte Fírimoin martaine nurunen.");  
}
```



# Reťazce a kódovanie





# Reťazce a kódovanie

- Trieda `std::string` je v skutočnosti iba jednou z realizácií šablóny triedy `std::basic_string` pre typ `char`
- Okrem nej existujú aj realizácie pre ďalšie typy, ktoré je možné použiť na reprezentáciu znakov
- Všetky triedy majú rovnaké rozhranie (dané šablónou), líšia sa iba v type poľa, v ktorom uchovávajú reťazec

```
std::string  
std::wstring  
std::u8string (C++20)  
std::u16string (C++11)  
std::u32string (C++11)
```

```
std::basic_string<char>  
std::basic_string<wchar_t>  
std::basic_string<char8_t>  
std::basic_string<char16_t>  
std::basic_string<char32_t>
```

# Úskalia práce s rôznymi kódovaniami

- Ak pre použité kódovanie platí, že **jeden code point je reprezentovaný jedným code unit**, môžeme s reťazcom *v našom programe\** pracovať prakticky bez zmien a špeciálnych opatrení
  - Medzi tieto kódovania patria:
    - Kódovanie ASCII (vrátane rozšírených) – code unit je reprezentovaný typom `char`
    - Kódovanie UTF-32 – code unit je reprezentovaný typom `char32_t` alebo, v systéme Linux a MacOS, typom `wchar_t`
- Metódy triedy (`basic_string` ako napríklad `operator[]`, `size`, `substr` budú fungovať podľa očakávaní

Problém môže nastať pri interakcii s okolím. \*

# Úskalia práce s rôznymi kódovaniami

- Ak použité kódovanie môže používať na **reprezentáciu jedného code point viac code unit** je potrebné zohľadniť, že metódy triedy (`basic_string`) pracujú s prvkami poľa reprezentujúcimi reťazec (code units)
- Obzvlášť nebezpečné sú metódy `operator[]` a `substr`, ktoré môžu vrátiť znak/reťazec, ktorý v danom kódovaní nie je platný
- Na druhej strane, napríklad porovnanie reťazcov na (ne)rovnosť metódou `operator==` bude fungovať bez zmeny\*

```
int main() {  
    std::u8string str(u8"🐼"); // 4  
    std::cout << str.size() << "\n";  
}
```

```
int main() {  
    std::string str("🐼"); // 4  
    std::cout << str.size() << "\n";  
}
```

Typ `char8_t` sa zatiaľ v praxi zatiaľ moc neujal a aj na reprezentáciu UTF-8 reťazcov sa často používa typ `char`.

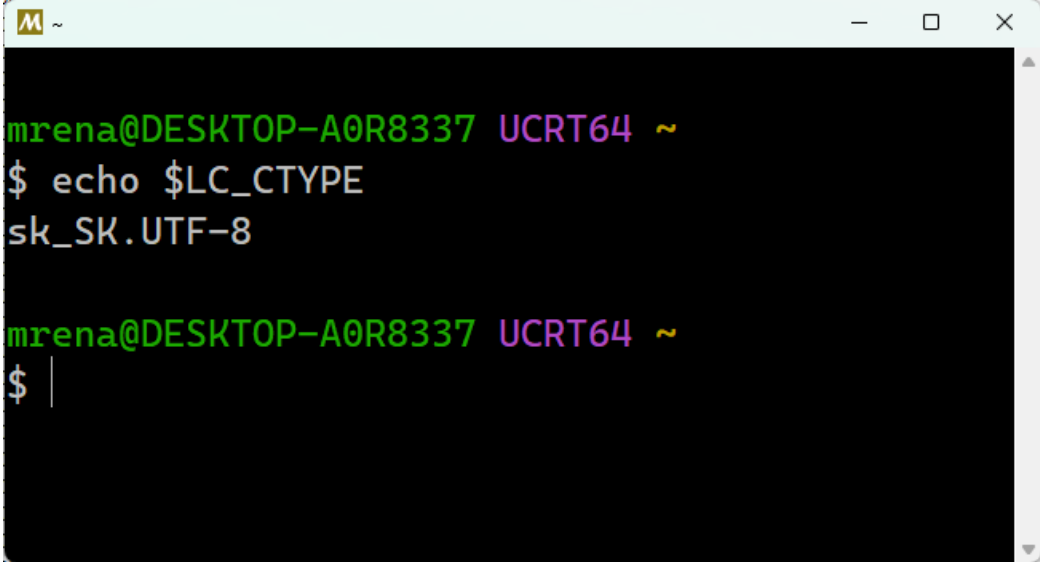
# Kódovanie a okolie

- Pre zabezpečenie správneho fungovania nášho programu je potrebné zohľadniť nielen kódovanie reťazcov, ale aj kódovanie vstupov a výstupov



# Terminál

- Predvolené kódovanie závisí na použitom termináli (okno, v ktorom beží interpreter)
- Prostredie UCRT64 (terminál [mintty](#) + interpreter [Bash](#)) používa UTF-8
- Novší [Windows Terminal](#) používa UTF-8
- Terminály v systémoch Linux a MacOS používajú UTF-8

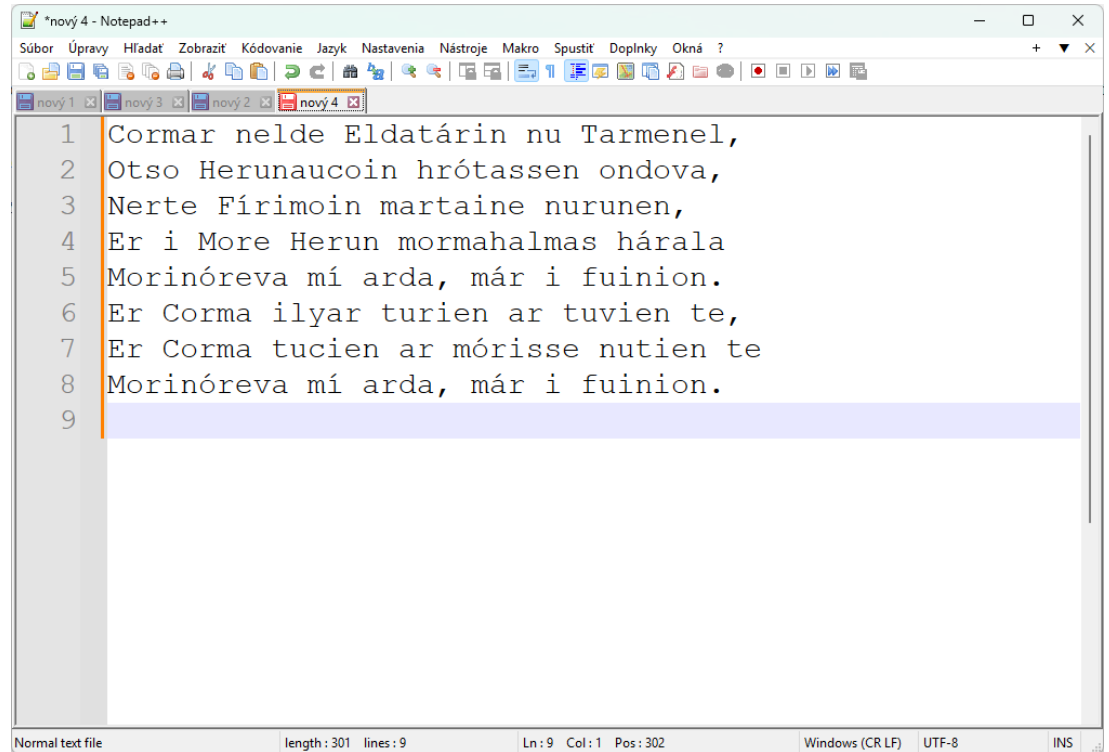


```
mrena@DESKTOP-A0R8337 UCRT64 ~  
$ echo $LC_CTYPE  
sk_SK.UTF-8  
  
mrena@DESKTOP-A0R8337 UCRT64 ~  
$ |
```

Príkazový riadok a starší PowerShell používajú rozšírené ASCII. Ich kódovanie je však možné [zmeniť](#).

# Súbory

- Pri práci so súborom je potrebné poznať kódovanie, ktoré je v ňom použité
- Táto informácia môže byť súčasťou metadát, dokumentácie alebo je len v povedomí programátora
- Kódovanie je možné detegovať automaticky, avšak tento prístup nemusí vždy fungovať



```
1 Cormar nelde Eldatárin nu Tarmenel,  
2 Otso Herunaucoin hrótassen ondova,  
3 Nerte Fírimoin martaine nurunen,  
4 Er i More Herun mormahalmas hárala  
5 Morinóreva mí arda, már i fuinion.  
6 Er Corma ilyar turien ar tuvien te,  
7 Er Corma tucien ar mórissime nutien te  
8 Morinóreva mí arda, már i fuinion.  
9
```

Na konverziu kódovania súborov je dobré použiť nástroj [iconv](#).

# Zdrojový kód

- Súbor so zdrojovým kódom nášho programu musí byť tiež (ako každý iný textový súbor) uložený v nejakom kódovaní
- Väčšina moderných programovacích jazykov podporuje UTF-8
- Kódovanie sa najviac dotýka **literálov**, ale je relevantné aj pri **identifikátoroch**
- Pre niektoré je UTF-8 predvoleným kódovaním
  - Tu hrá dôležitú úlohu spätná kompatibilita s kódovaním ASCII

```
g++ -std=c++20 -finput-charset=utf-8 -fexec-charset=utf-8 main.cpp
```

```
cl /utf-8 main.cpp # MSVC
```

```
void (std::string_view str) {  
    std::cout << str << "\n";  
}  
  
int main() {  
    ("Hello world!");  
}
```

# Štandardný vstup a výstup





# Štandardný vstup

- **Štandardný vstup** je reprezentovaný globálnou premennou `std::cin`
- Typ tejto premennej je `std::istream`, ktorý vo všeobecnosti reprezentuje **vstupný prúd znakov (bajtov)**
- Premenná je deklarovaná v hlavičke `<iostream>`
- Rozhranie triedy `istream` je z veľkej časti postavené na preťažení operátora `>>` pre načítanie (extrakciu z prúdu bajtov) všetkých relevantných typov

```
int main() {  
    std::string word;  
    int number;  
    double otherNumber;  
    std::cin >> word;  
    std::cin >> number;  
    std::cin >> otherNumber;  
    if (std::cin.fail()) {  
        std::cerr << "Failed to load some input.\n";  
    }  
}
```

# Spracovanie vstupu

- Ak potrebujeme pri spracovaní vstupu väčšiu flexibilitu, môžeme načítať reťazec a ten následne konvertovať na požadovaný typ
- (od C++11) môžeme použiť funkcie na konverziu celých a reálnych čísel
- Tieto funkcie nám umožňujú lepšie identifikovať chyby pri konverzii

```
int main() {  
    std::string numStr;  
    std::cin >> numStr;  
    size_t charsProcessed;  
    int num = std::stoi(numStr, &charsProcessed, 10);  
    if (charsProcessed != numStr.size()) {  
        std::cerr  
            << numStr  
            << " is not a valid number!\n";  
    }  
}
```

# Spracovanie vstupu

- Ak potrebujeme pri spracovaní vstupu väčšiu flexibilitu, môžeme načítať reťazec a ten následne konvertovať na požadovaný typ
- (od C++17) môžeme použiť funkcie z hlavičky `<charconv>`
- Ich rozhranie je nízkoúrovňovejšie (pracujú s reťazcami v štýle jazyka C) a mali by byť rýchlejšie
- Poskytuje detailnejší popis výsledku konverzie

```
int main() {  
    std::string numStr;  
    std::cin >> numStr;  
    int num;  
    std::from_chars_result status = std::from_chars(  
        numStr.c_str(),  
        numStr.c_str() + numStr.size(),  
        num,  
        10);  
    if (status.ptr != numStr.c_str() + numStr.size()) {  
        std::cerr  
            << numStr  
            << " is not a valid number!\n";  
    }  
}
```

# Pokročilé spracovanie vstupu

- V situáciách, kedy je vstup jasne štruktúrovaný (je možné popísať ho gramatiokou) používame na jeho načítanie parser
  - Príkladom takéhoto vstupu je zdrojový kód programovacieho jazyka
- Parser je funkcia, ktorej vstupom je štruktúrovaný text a výstupom je údajový typ z typového systému jazyka, v ktorom parser spúšťame (alebo informácia o chybe, ak vstup nie je v správnom formáte)
- Parser generujeme na základe gramatiky s použitím generátora parserov (knižnica):
  - Lexy
  - Boost.Spirit
  - Tree Sitter

# Pokročilé spracovanie vstupu

```
<IPv4-address> ::= <decimal-octet> "." <decimal-octet> "." <decimal-octet> "." <decimal-octet>

<decimal-octet> ::= "0"
                  | <nonzero-digit>
                  | <nonzero-digit> <digit>
                  | "1" <digit> <digit>
                  | "2" <zero-to-four>
                  | "25" <zero-to-five>

<digit>          ::= "0" | <nonzero-digit>
<nonzero-digit> ::= "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
<zero-to-four>  ::= "0" | "1" | "2" | "3" | "4"
<zero-to-five>  ::= "0" | "1" | "2" | "3" | "4" | "5"
```

Gramatika IPv4 adresy v Backus–Naur forme.

# Pokročilé spracovanie vstupu

```
// Parse an IPv4 address into a `std::uint32_t`.
struct ipv4_address {
    // What is being matched.
    static constexpr auto rule = []{
        // Match a sequence of (decimal) digits and convert it into a std::uint8_t.
        auto octet = dsl::integer<std::uint8_t>;
        // Match four of them separated by periods.
        return dsl::times<4>(octet, dsl::sep(dsl::period)) + dsl::eof;
    }();

    // How the matched output is being stored.
    static constexpr auto value = lexy::callback<std::uint32_t>(
        [](std::uint8_t a, std::uint8_t b, std::uint8_t c, std::uint8_t d) {
            return (a << 24) | (b << 16) | (c << 8) | d;
        });
};
```

Parser pre IPv4 adresy v knižnici lexy.

# Štandardný výstup

- Program má tri štandardné výstupy reprezentované globálnymi premennými typu `std::ostream` (**výstupný prúd znakov (bajtov)**):
  - `std::cout` – štandardný výstup
  - `std::cerr` – štandardný chybový výstup
  - `std::clog` – štandardný logovací výstup
- Premenná sú deklarované v hlavičke `<iostream>`
- Rovnako ako pri vstupe je práca s výstupnými prúdmi z veľkej časti postavená na preťažení operátora `<<` pre zápis (do prúdu bajtov) všetkých relevantných typov

```
int main() {  
    std::string name = "Larry";  
    double e = 2.71;  
    int x = 20;  
    std::cout << name << "\n";  
    std::cout << e << "\n";  
    std::cout << x << "\n";  
}
```

# Formátovanie výstupu

- Formátovanie výstupu môžeme ovplyvniť manipulátormi prúdu
- Môžeme nimi nastaviť napríklad počet desatinných miest alebo pridanie znakov pre dosiahnutie požadovanej veľkosti výstupu
- Manipulátory používame ich zapísaním do výstupného prúdu

```
int main() {  
    double sq2 = std::sqrt(2);  
    std::cout  
        << std::setprecision(4)  
        << std::setw(10)  
        << std::setfill('~');  
    std::cout << sq2 << "\n";  
    // ~~~~1.414  
}
```



# Formátovanie výstupu

- Operátor << môžeme preťažiť pre nami definovaný typ
- Inštancie nášho typu potom môžeme zapisovať priamo do výstupného prúdu

```
std::ostream &operator<<(
    std::ostream &ost,
    const Person &p
) {
    ost << "[name=" << p.getName() << "]\n";
    return ost;
}

int main() {
    Person p("Larry");
    std::cout << p << "\n";
    // [name=Larry]
}
```

# Alternatívne formátovanie výstupu

- Manipulátory výstupu sú pomerne mnohoslovné (angl. verbose)
- (od C++20) Modernejším prístupom je použitie funkcie `std::format`
- Funkcia pochádza z knižnice fmt, v štandardnej knižnici ju definuje hlavička `<format>`
- (od C++23) Rovnaké formátovanie môžeme použiť aj priamo vo výpise použitím funkcie `std::print` v štandardnej knižnici ju definuje hlavička `<print>`

```
int main() {  
    double sq2 = std::sqrt(2);  
    std::cout  
        << std::format("{:~>10.4g}\n", sq2);  
}
```

```
int main() {  
    double sq2 = std::sqrt(2);  
    std::print("{:~>10.4g}\n", sq2);  
}
```

Podpora funkcie `print` však ešte nie je moc rozšírená.

# Lokalizácia

- Ďalšou vecou, ktorá ovplyvňuje formát výstupu je aktuálne nastavenie lokalizácie
- V jazyku C++ je lokalizácia reprezentovaná triedou `std::locale`
- Predvolená hodnota sa preberie z nastavenia systému
- Lokalizácia ovplyvňuje napríklad výpis a načítanie:
  - dátumu
  - mien (peňazí)
  - oddeľovač desatinných miest a tisícov

# Nastavenie lokalizácie

- Lokalizáciu je možné nastaviť globálne alebo samostatné pre každý vstupný výstupný/prúd
- Vybraná lokalizácia však musí byť nainštalovaná v systéme

```
int main() {  
    std::locale::global(std::locale("sk_SK.utf8"));  
    std::cout << 2.71 << "\n";  
}
```

```
int main() {  
    std::cout.imbue(std::locale("sk_SK.utf8"));  
    std::cout << 2.71 << "\n"; // 2,71  
    std::cout.imbue(std::locale("en_EN.utf8"));  
    std::cout << 2.71 << "\n"; // 2.71  
}
```

# Práca so súbormi

- So súbormi pracujeme pomocou rozhrania tried vstupných a výstupných prúdov
- Na zápis používame triedu `std::ofstream` a na čítanie triedu `std::ifstream`
- Obe triedy poskytuje hlavička `<fstream>`

```
int main() {  
    double sq2 = std::sqrt(2);  
    std::ofstream ofst(  
        "number.txt",  
        std::ios_base::out);  
    if (!ofst.is_open()) {  
        std::cerr << "Failed to open number.txt\n";  
        return 1;  
    }  
    ofst  
        << std::setprecision(4)  
        << std::setw(10)  
        << std::setfill('~');  
    ofst << sq2 << "\n";  
}
```

Trieda `ofstream` v konštruktore súbor otvorí a v deštruktore **(automaticky) zatvorí**. Ide o ďalšiu ukážku sily návrhového vzoru **RAII**.

# Vstupný a výstupný buffer

- Triedy `std::istream` a `std::ostream` na pozadí používajú triedu `std::streambuf`, ktorá poskytuje nízkoúrovňový prístup k sekvencii znakov (bajtov)
- Úlohou prúdov je formátovanie a konverzie, samotný zápis a načítavanie bajtov zabezpečuje trieda `std::streambuf`
- Trieda `std::streambuf` spravuje aj tzv. **vstupný/výstupný buffer** – pole znakov preddefinovanej veľkosti
- Nie všetko, čo do výstupného prúdu zapíšeme, sa hneď objaví vo výstupe, ktorý prúd reprezentuje (terminál, súbor, ...)
- Toto správanie závisí od typu výstupu

# Štandardný výstup

- Štandardný výstup (`std::cout`) je typicky riadkovo buffrovaný
- Buffer sa na výstup zapíše, keď sa do výstupu zapíše **znak pre nový riadok**
- Znak pre nový riadok sa pre jednotlivé platformy líši:
  - Pre unixové systémy (Linux, macOS) je to znak `'\n'`
  - Pre systém Windows je to postupnosť dvoch znakov `'\r'` a `'\n'`
- Znak `'\n'` sa nazýva Line Feed (LF), ASCII aj Unicode hodnota 10
- Znak `'\r'` sa nazýva Carrige Return (CR), ASCII aj Unicode hodnota 13
- Trieda `std::ostream` **automaticky vloží platformovo špecifické ukončenie riadku** pri zápise znaku `'\n'`

# Štandardný výstup

- Buffer štandardného výstupu (`std::cout`) sa zapíše aj v prípade, že niečo načítame zo štandardného vstupu (`std::cin`)
- Buffer štandardného výstupu (`std::cout`) sa zvyčajne zapíše aj v prípade, že niečo zapíšeme do štandardného chybového výstupu (`std::cerr`) (takéto prúdy označujeme ako previazané)
- Automaticky sa tiež zapíše na konci programu
- Štandardný chybový výstup (`std::cerr`) nie je buffrovaný, všetko čo do neho zapíšeme sa hneď vypíše (niektoré terminály môžu chybový výstup zobrazovať napríklad inou farbou)
- Štandardný logovací výstup (`std::clog`) je buffrovaný, podobne ako štandardný výstup



# Súbory

- Pri zápise do súboru typicky sa buffer zapíše až po jeho naplnení
- Operáciu zápisu buffra označujeme angl. ako flush
- O zápis buffra je možné explicitne požiadať funkciou `flush`
- Časté zapisovanie do súboru **môže byť pomalé** (preto sa buffer zapíše, až keď sa naplní) – chceme sa mu vyhnúť

```
int main() {
    double sq2 = std::sqrt(2);
    std::ofstream ofst(
        "number.txt",
        std::ios_base::out);
    if (!ofst.is_open()) {
        std::cerr << "Failed to open number.txt\n";
        return 1;
    }
    ofst
        << std::setprecision(4)
        << std::setw(10)
        << std::setfill('~');
    ofst << sq2 << "\n";
    ofst.flush();
    ofst << sq3 << "\n";
}
```

# Súbory

- Na explicitný zápis (flush) buffra môžeme použiť aj manipulátory prúdu `std::flush` a `std::endl`
- `std::flush` má rovnaký efekt ako zavolanie funkcie `flush`
- `std::endl` má rovnaký efekt ako zavolanie funkcie `flush`, ale predtým do prúdu vloží ukončenie riadku

```
int main() {  
    double sq2 = std::sqrt(2);  
    std::ofstream ofst(  
        "number.txt",  
        std::ios_base::out);  
    if (!ofst.is_open()) {  
        std::cerr << "Failed to open number.txt\n";  
        return 1;  
    }  
    ofst  
        << std::setprecision(4)  
        << std::setw(10)  
        << std::setfill('~');  
    ofst << sq2 << std::endl;  
    ofst << sq3 << "\n" << std::flush;  
}
```

# Manipulátor `std::endl`

- Manipulátor `std::endl` sa často chybne používa v domnienke, že vkladá platformovo špecifické ukončenie riadku
- Jeho vloženie však zabezpečí trieda `std::ostream` automaticky\*
- Preto, na ukončenie riadku používajte znak `'\n'`
- Na zápis buffra používajte funkciu (členskú) `flush` alebo manipulátor `std::endl`
- Použitie manipulátora `std::endl` na ukončovanie riadkov môže spôsobiť **častý a pomalý zápis do súboru**

Ak nepracujeme s binárnym súborom. \*

# Zhrnutie

- Je potrebné mať prehľad o rôznych kódovaniach
  - UTF-8 je v mnohých oblastiach prakticky štandard
  - Je dôležité vedieť o kódovaní
    - Vstupov programu
    - Programu samotného (literály a reťazce)
    - Výstupu
- Na prácu so vstupom a výstupom používame vstupné a výstupné prúdy, ktoré (okrem iného) môžu reprezentovať terminál alebo súbor
  - Je potrebné zohľadňovať mechanizmus buffrovania

# Ďakujem za pozornosť

Priestor na otázky

